```
rm(list = ls(all = TRUE))
#Do not delete this!
#It clears all variables to ensure reproducibility
```

```
library(INLA)
```

```
## Loading required package: Matrix
```

```
## This is INLA_24.12.11 built 2024-12-11 19:58:26 UTC.
##  - See www.r-inla.org/contact-us for how to get help.
##  - List available models/likelihoods/etc with inla.list.models()
##  - Use inla.doc(<NAME>) to access documentation
```

MODELLING RAINFALL IN EDINBURGH, USING INLA

This problem looks at modelling rainfall in Edinburgh.

```
# Load data
Edi.rain <- read.csv("Edinburgh_rainfall.csv")
head(Edi.rain)
```

```
##   rainfall windspeed temperature     year
## 1 1.239984  1.313964    272.0122 1940.000
## 2 2.902047  1.651040    275.0046 1940.083
## 3 2.412593  2.863953    277.3972 1940.167
## 4 3.493554  1.250086    280.3240 1940.250
## 5 1.249396  1.378372    284.0099 1940.333
## 6 1.388603  2.024482    287.8907 1940.417
```

The dataset consists of 12*83=996 monthly observations of different environmental variables sampled in Edinburgh. These variables includes:

- daily rainfall total (mm)
- average windspeed (m/s)
- average temperature (K)
- year: (this is stored as a real number between 1940 and 2023, i.e. 1950.0 and 1950.5 correspond to January and July for 1950.)

All environmental variables are provided as monthly averages (so rainfall is recorded as the monthly average of the daily total rainfall). Note that the last 12 observations of rainfall, corresponding to the year 2022, are missing (set to NA).

```
#Deal with missing data from 2022.
print(nrow(Edi.rain))
```

```
## [1] 996
```

```
print(sum(is.na(Edi.rain$rainfall)))
```

```
## [1] 12
```

The number of missing data of the response is 12 (it's only the data from the 2022 that are missing). We do not consider these data while building our initial regression model.

```
#create a dataset without NAs
Edi.rain.clean <- na.omit(Edi.rain)
```

```
#standardise the covariates
Edi.rain.clean$windspeed_s  <- scale(Edi.rain.clean$windspeed,
                                     center=TRUE, scale=TRUE)

Edi.rain.clean$temperature_s <- scale(Edi.rain.clean$temperature,
                                      center=TRUE, scale=TRUE)

Edi.rain.clean$year_s  <- scale(Edi.rain.clean$year, center=TRUE, scale=TRUE)

#setting priors
prec_prior <- list(prec=list(prior='loggamma',param = c(1,0.01) ) )

beta_prior <- list(mean.intercept = 0, prec.intercept = 1/(100^2),
                   mean =0 , prec=1/(100^2))

#fit model in INLA
mod_rainfall_lin_inla <- inla(rainfall ~ windspeed_s + temperature_s + year_s,
                              data= Edi.rain.clean, family='gaussian',
                              control.family = list(hyper=prec_prior),
                              control.fixed = beta_prior,
                              control.compute = list(cpo=TRUE, waic=TRUE),
                              verbose=TRUE)
```

First, the covariates that are numerical are scaled (centered and standardised), to allow a better performance of the Bayesian model and a easier interpretation of the posterior results. In our model all covariates are scaled.

The priors are set, using a prior inverse-Gamma for the variance, that is equivalent to store a Gamma prior on τ (the precision). To do that, since INLA internally stores the log of the precision, we set a log-gamma prior (parameters 1 and 0.01 remain the same for log-gamma).

For the Normal prior of the coefficients β, we set mean $\mu_0 = 0$ and INLA requires precision as argument : τ_0=1/ σ_0^2 , then τ_0= 1/(100^2), where 100 is the requested standard deviation σ_0.

We call the INLA function, giving to it the formula with scaled covariates: Year, Temperature and Windspeed. The priors for β are passed through control.fixed, and for the τ through control.family.

Our linear regression model has rainfall as response.

Remember that year is not a categorical variable, but a numerical one.

control.compute =list(cpo=TRUE, waic=TRUE) is used to compute the residuals later, in order to evaluate the goodness of fit.

```
summary(mod_rainfall_lin_inla)
```

```
## Time used:
##     Pre = 0.241, Running = 0.476, Post = 0.0444, Total = 0.761
## Fixed effects:
##                mean    sd 0.025quant 0.5quant 0.975quant  mode kld
```

```
## (Intercept)   2.236 0.031      2.176    2.236      2.297 2.236   0
## windspeed_s   0.081 0.032      0.019    0.081      0.143 0.081   0
## temperature_s 0.027 0.032     -0.035    0.027      0.089 0.027   0
## year_s        0.154 0.031      0.092    0.154      0.215 0.154   0
##
## Model hyperparameters:
##                                          mean    sd 0.025quant 0.5quant
## Precision for the Gaussian observations 1.06 0.048      0.967     1.06
##                                          0.975quant mode
## Precision for the Gaussian observations       1.15 1.06
##
## Watanabe-Akaike information criterion (WAIC) ...: 2744.95
## Effective number of parameters .................: 5.27
##
## Marginal log-Likelihood:  -1406.36
## CPO, PIT is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

Keeping in mind that the covariates have been scaled, the coefficient β_i for one covariate corresponds to how the response changes for one standard deviation of the considered covariate i.

The mean precision for the Gaussian observation is 1.06, that means that the mean variance of the residuals is around $1/1.06$. The 95% Bayesian CI at 95% is $[0.967, 1.15]$, providing a quite stable estimate.

The intercept is the rain we expect when all covariates are at 0, it's value in this case is 2.236 (average daily rain we expect when temperature, windspeed and year are all 0).

CI at 95% for the $\beta_{temperature}$ coefficient includes 0, meaning that we do not expect temperature to have a statistically relevant role in the prediction of the response.

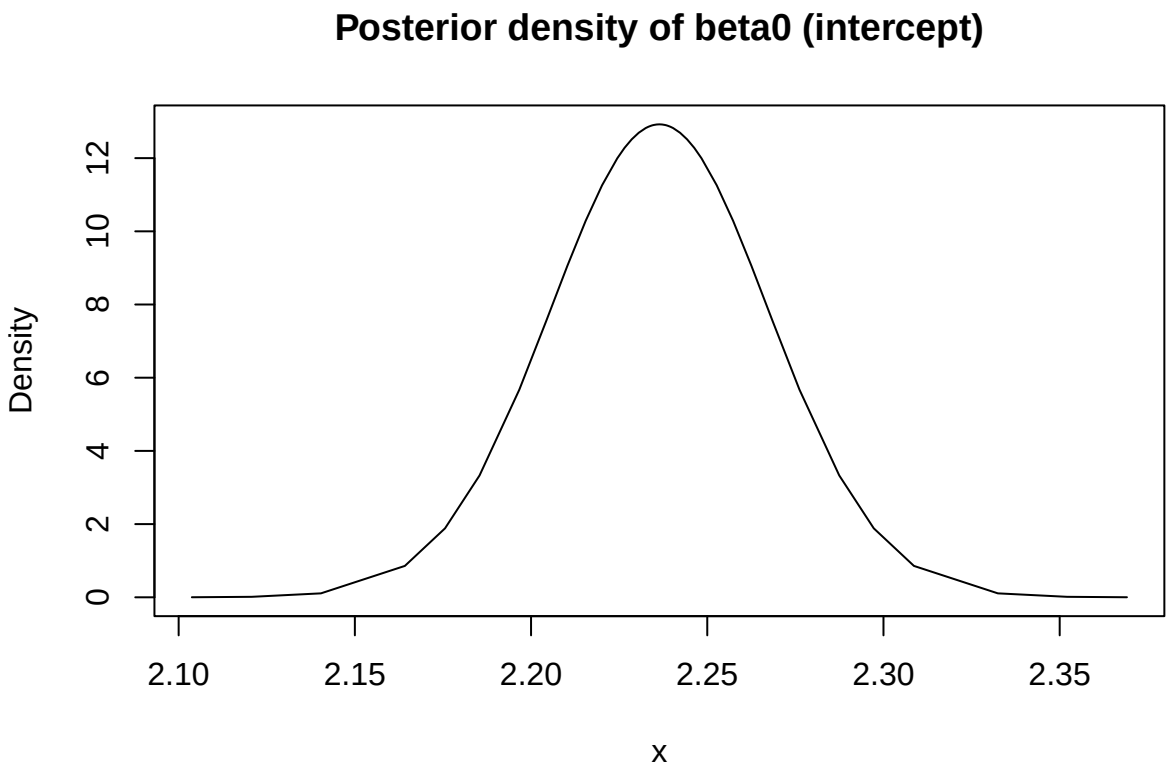
On the other hand speed of wind seems to have a more important role, posterior mean of $\beta_{windpseed} = 0.081$, and CI at 95% that do not include 0.

The mean posterior value for β_{year} is 0.154, meaning that in average daily rainfall keeps increasing over the years. So we expect rain increasing over time.

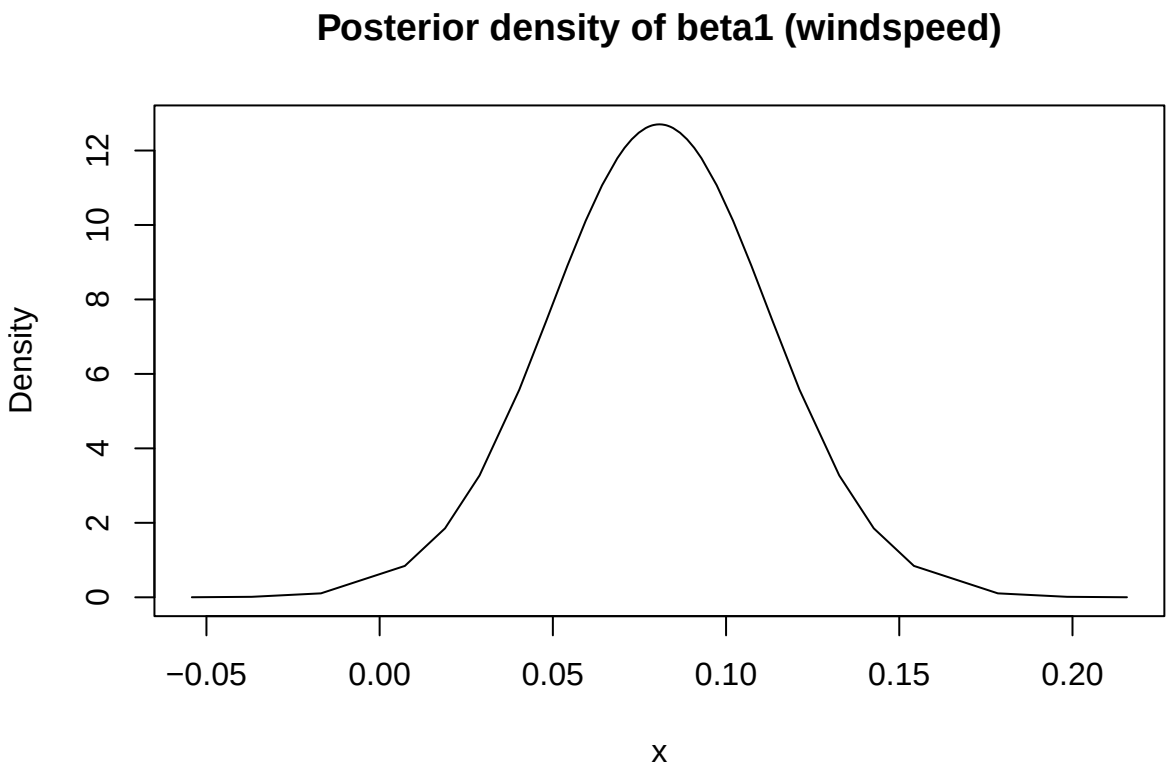
Since the covariates are all on the same scale, looking at mean values (and CIs) of the β coefficients, we can conclude that the most influential covariates are year and windspeed (the biggest mean posterior value is the one from year, but we cannot be statistically sure it's gonna be the greatest value since the CIs overlap for the two covariates).

Using the plot function it's possible to visualize the marginal posterior densities of the regression coefficients:

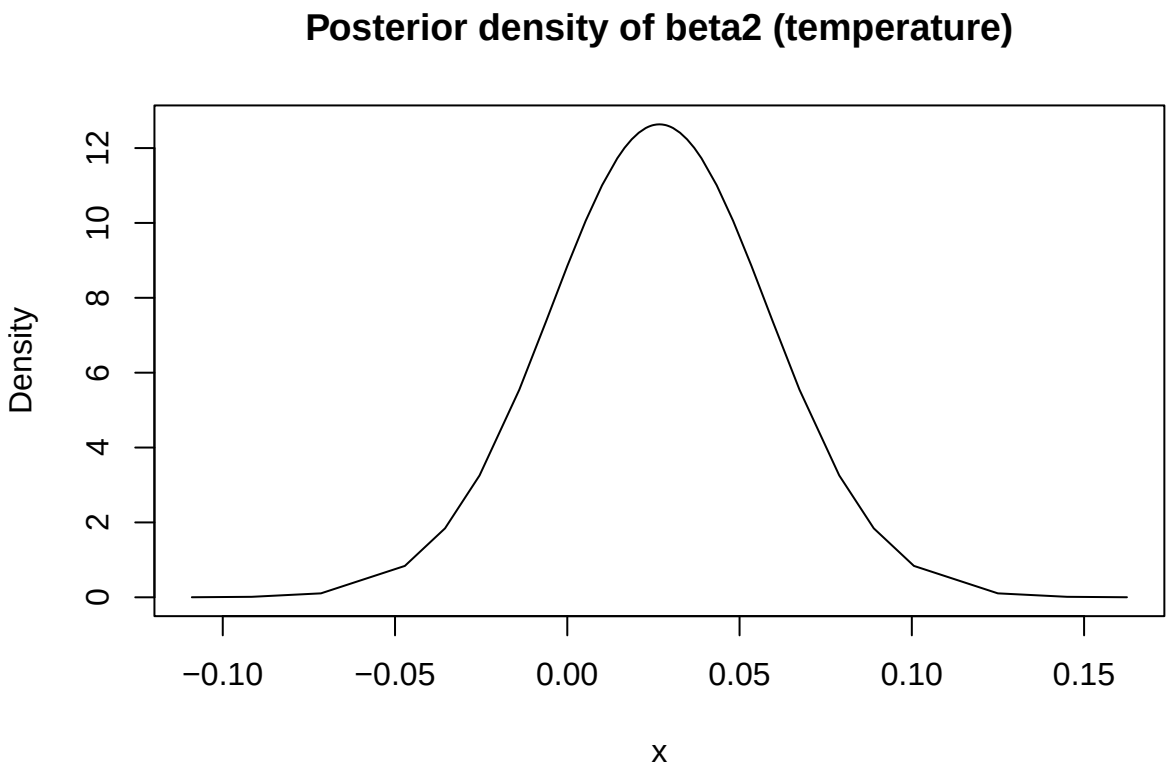
```
plot(mod_rainfall_lin_inla$marginals.fixed$'(Intercept)',type='l',xlab='x',
     ylab='Density',
     main ='Posterior density of beta0 (intercept)')
```



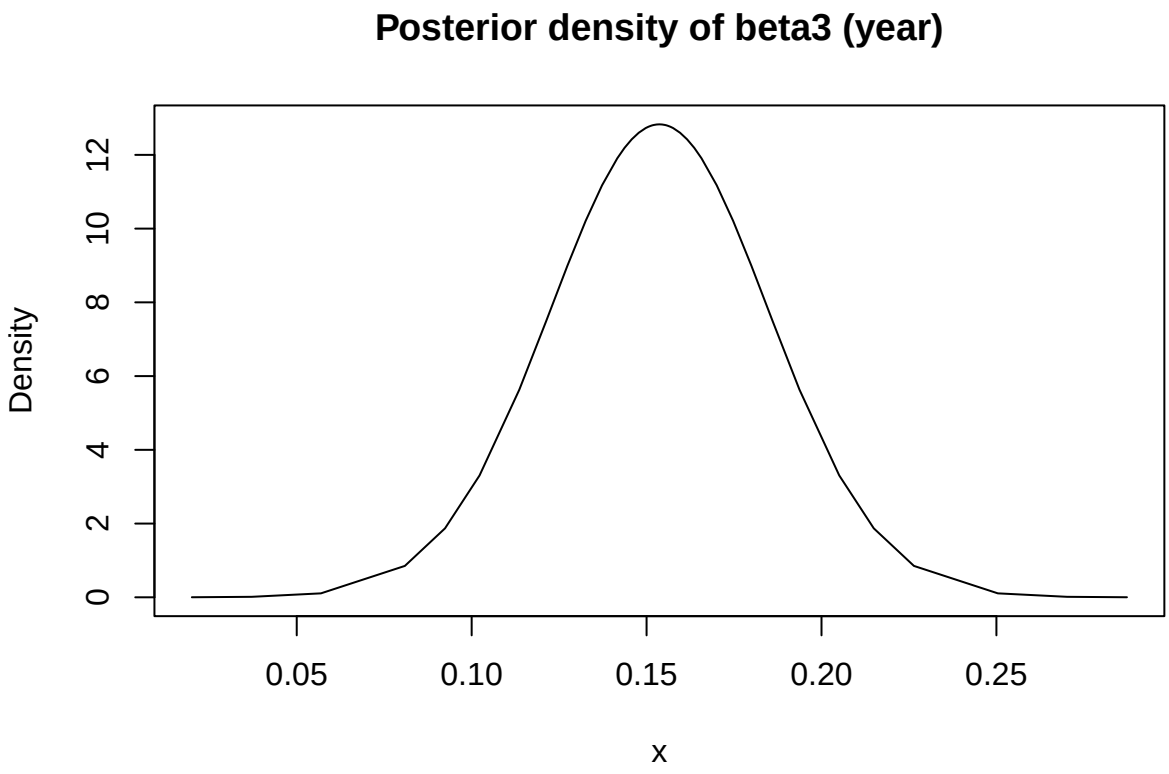

```
plot(mod_rainfall_lin_inla$marginals.fixed$'windspeed_s',type='l',xlab='x',
     ylab='Density',
     main ='Posterior density of beta1 (windspeed)')
```




```
plot(mod_rainfall_lin_inla$marginals.fixed$'temperature_s',type='l',xlab='x',
     ylab='Density', main ='Posterior density of beta2 (temperature)')
```




```
plot(mod_rainfall_lin_inla$marginals.fixed$'year_s',type='l',xlab='x',
     ylab='Density',
     main ='Posterior density of beta3 (year)')
```



The posterior density plots show the estimated distributions of the regression coefficients β in the INLA model.

The unimodal nature of these distributions suggests that the inference process worked well.

As previously stated the $\beta_{temperature}$ distribution is quite centered around 0, making it's role in the regression model not much relevant. Also, the $\beta_{windspeed}$ distribution shows in it's domain the value 0, but only in the left tail, meaning that it's not highly likely for windspeed not to have an influence in the model (as previuosly noted the CIs for windspeed do NOT contain 0).

```
#Fit model to sample from the posterior
mod_rainfall_lin_inla_post <- inla(rainfall ~ windspeed_s
                                   + temperature_s + year_s,
                                   data=Edi.rain.clean, family='gaussian',
                                   control.family = list(hyper=prec_prior),
                                   control.fixed = beta_prior,
                                   control.compute = list(config=TRUE),
                                   verbose=TRUE)
```

```
nsamp <- 5000 #SET LOW FOR COMPUTATIONAL FEASABILTIY

#Sampling
rainfall_samples <- inla.posterior.sample(n=nsamp,
                                          result=mod_rainfall_lin_inla_post)

#Get fitted values
fitted_values <- inla.posterior.sample.eval(function(...) {Predictor},
```

```
                                          rainfall_samples )

#Get sigma squared samples (sigma2= 1/tau)
sigma2 <- 1/(inla.posterior.sample.eval(function(...) {theta},rainfall_samples))

#Studentized residuals

n <- nrow(Edi.rain.clean)
x <- cbind(rep(1,n), Edi.rain.clean$windspeed_s, Edi.rain.clean$temperature_s,
           Edi.rain.clean$year_s ) #design matrix

H <- x %*% solve((t(x) %*% x)) %*% t(x) #hat matrix

y <-Edi.rain.clean$rainfall

studentised_res <- matrix(0, nrow=n, ncol=nsamp)

for(l in 1:nsamp){
  for(i in 1:n){
    studentised_res[i,l] <- (y[i] - fitted_values[i,l]) /
      (sqrt(sigma2[l] * (1-diag(H)[i])))
  }
}

#posterior mean of studentised residuals
stude_res_mean <- apply(studentised_res , 1, mean)
```

The inla model is fitted again, this time using "control.compute=list(config=TRUE)" and "control.predictor=list(compute=TRUE)" in order to get samples from the posterior, that are used to compute the studentised residuals.

In particular, the fitted values have been sampled using the function to extract the linear predictiors, this works since in this model the link function is the identity function, so the fitted values are the linear predictors.

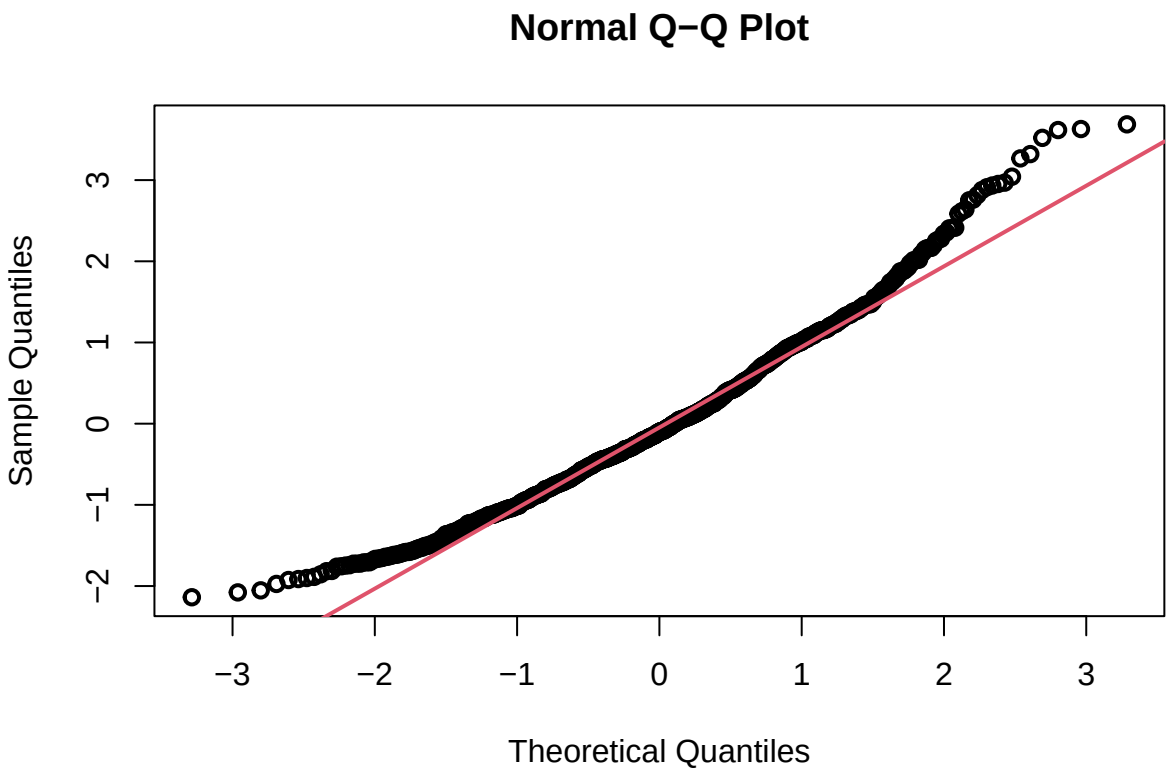
In order to get the studentised residuals, the sample from σ^2 are needed as well: the posterior samples are computed for τ (hyperparameter theta) and then $\sigma^2 = 1/\tau$.

The number of samples has been setted to $n_{samp} = 5000$.

Later on, the studentised residuals are computed in a matrix of dimension $(n \times n_{samp})$, where n is number of observations in the original dataset and n_{samp} is the number of samples.

For each observation in each sample, the studentised residual is computed and stored into the matrix of the residuals. In order to get the posterior mean of the residuals for each observation the function apply is used on the rows, taking the mean of all the studentised residuals of all the samples for each observation. We end up with a vector of length n, containing the posterior means of the studentised residuals.

```
#QQ-plot
qqnorm(stude_res_mean, lwd=2)
qqline(stude_res_mean, col=2, lwd=2)
```

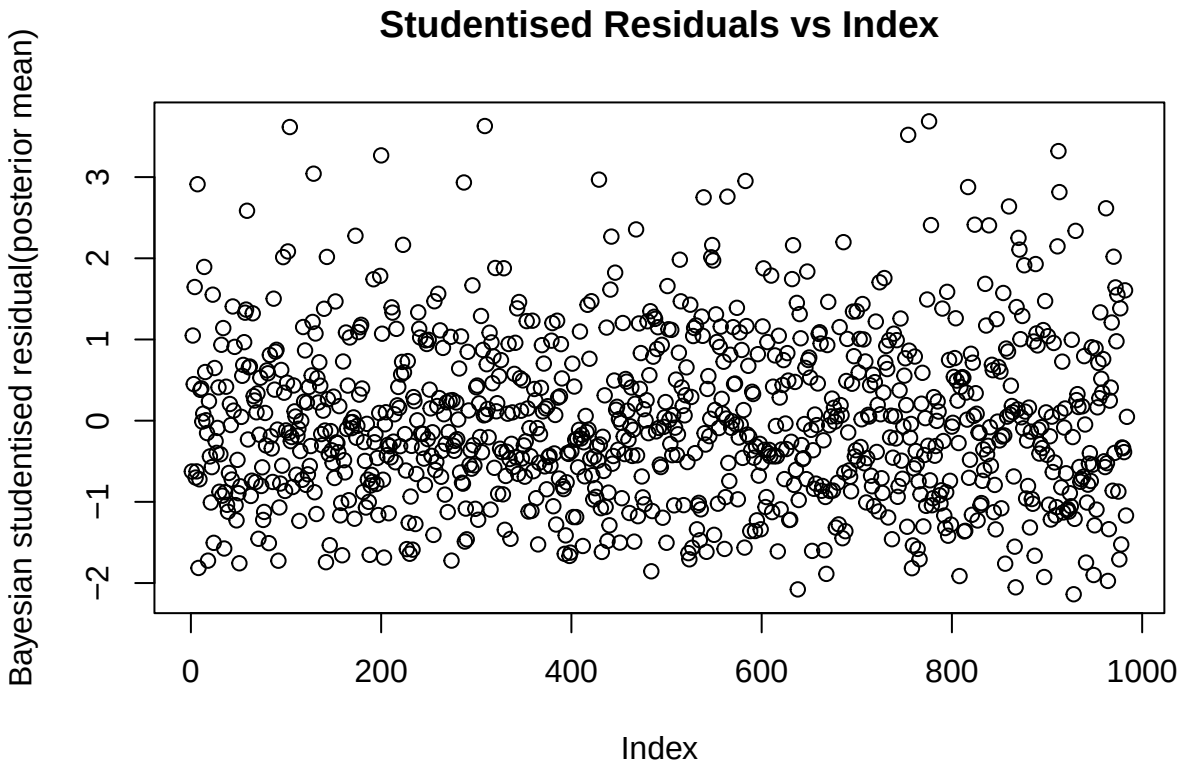


This Normal QQ-Plot evaluates the normality of residuals by comparing sample quantiles to theoretical quantiles. The points should stay on the red reference line, but there are deviations in the tails, suggesting heavy-tailed residuals.

This implies that a Gaussian model is not probably the best fist, a robust regression approach might improve the model.

```
#Studentized res vs index

plot(1:n, stude_res_mean, xlab='Index',
     ylab='Bayesian studentised residual(posterior mean)',
     main = 'Studentised Residuals vs Index')
```



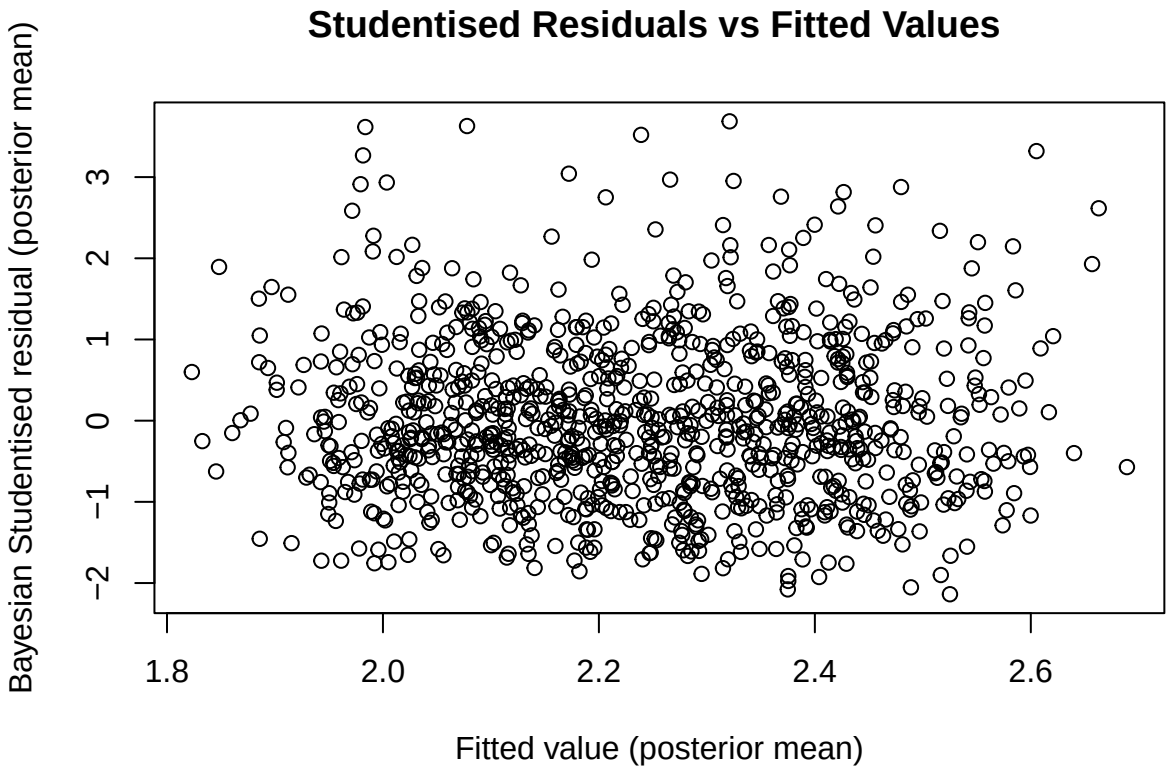
To check independence of the error terms, the studentised residuals are plotted against the index. No particular patterns is shown in the plot, meaning that the residuals look independent. Also, the residuals look evenly spread meaning that the assumption of constant variance seems to hold. Some outliers (big residual compare to other observation) can be noticed.

```
#Studentised residuals vs Fitted Values

#Compute posterior mean fitted values

fit_val_mean <- apply(fitted_values,1,mean)

plot(fit_val_mean,stude_res_mean,xlab="Fitted value (posterior mean)",
     ylab="Bayesian Studentised residual (posterior mean)",
     main="Studentised Residuals vs Fitted Values ")
```



Studentised Residuals are plotted against Fitted Values. Both independence and constant variance hypothesis seem to hold, again some outliers are visible.

Robust regression:

```
#Weibull
#inla.doc("weibull")

#Set prior for shape parameter

alpha_prior <-list(theta = list( prior = "loggamma",  # Log-Gamma prior on log(alpha)
      param = c(1, 0.01))) #non-informative prior!


#fit model in INLA
mod_rainfall_lin_wei <- inla(rainfall ~ windspeed_s + temperature_s + year_s,
                             data= Edi.rain.clean, family='weibull',
                             control.family = list(hyper= alpha_prior),
                             control.fixed = beta_prior,
                             control.compute = list(cpo=TRUE, waic=TRUE),
                             verbose=TRUE)
```

Using 'inla.doc("weibull")', it's possible to access the documentation for the Weibull parametirasation in INLA.

Given y distributed as Weibull(α, λ), the density is:

$$f(y) = \alpha y^{\alpha-1}\lambda exp(-\lambda y^{\alpha}), \alpha > 0, \lambda > 0,$$

(there are two possible parametirastions : default value is variant = 0, the one that we use).

The parameter λ is linked to the linear predictor as $\lambda = exp(\eta)$.

The hyperparameter is α, with $\alpha = exp(S\theta)$, the prior is defined on θ and S= 0.1 as default.

The prior chosen for the θ is log-Gamma(1,0.01), trying to be non-informative.

```
summary(mod_rainfall_lin_wei)
```

```
## Time used:
##     Pre = 0.149, Running = 0.71, Post = 0.0481, Total = 0.908
## Fixed effects:
##                  mean    sd 0.025quant 0.5quant 0.975quant   mode kld
## (Intercept)   -1.934 0.050     -2.032   -1.934     -1.835 -1.934   0
## windspeed_s   -0.052 0.032     -0.114   -0.052      0.010 -0.052   0
## temperature_s -0.047 0.034     -0.113   -0.047      0.019 -0.047   0
## year_s        -0.137 0.032     -0.200   -0.137     -0.074 -0.137   0
##
## Model hyperparameters:
##                               mean    sd 0.025quant 0.5quant 0.975quant mode
## alpha parameter for weibull 2.15 0.037       2.07     2.15       2.22 2.15
##
## Watanabe-Akaike information criterion (WAIC) ...: 2707.40
## Effective number of parameters .................: 3.85
##
## Marginal log-Likelihood:  -1400.32
## CPO, PIT is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

The β coefficients are flipped in sign, but this does not change the interpretation compared to the previous Gaussian model, since $\lambda = exp(\eta) = exp(\beta_0 + \beta_1 \times x_1 + \beta_2 \times x_2 + \beta_3 \times x_3)$.

Given y distributed as a Weibull , λ in the density governs how fast the decay of the density happens for high values of y.

Then, if β_i increases the λ increases as well, but given the Weibull density, where higher λ means lower probability of large y (response) and it's more probable to have low values of y. On the other hand, if β_i decreases increased values of y are more likely to be seen.

Similarly to the normal model: Year is the most significant covariate and Temperature is not statistically significative, on the other hand also windspeed in Weibull seems not relevant.

Now let's work on a model using the gamma family:

```
#Gamma
#inla.doc("gamma")

prec_prior <-list(theta = list( prior = "loggamma",  # Log-Gamma prior on log(alpha)
      param = c(1, 0.01))) #non-informative prior!


#fit model in INLA
```

```
mod_rainfall_lin_gamma <- inla(rainfall ~ windspeed_s + temperature_s + year_s,
                               data= Edi.rain.clean, family='gamma',
                               control.family = list(hyper= alpha_prior),
                               control.fixed = beta_prior,
                               control.compute = list(cpo=TRUE, waic=TRUE),
                               verbose=TRUE)
```

Explanation

Using 'inla.doc("gamma")', it's possible to access the documentation for the Gamma parametirasation in INLA.

Given y distributed as $Gamma(a, b)$, then:

$$f(y) = \frac{1}{\Gamma(a)} y^{(a-1)} exp(-by), a > 0, b > 0, y > 0,$$

where shape=a, scale = $1/b$.

The hyperparameter is the precision ϕ, where:

$$b^2/a = (s\phi)/\mu^2 and \mu = a/b$$

Where s is a constant with default value 1. Then, solving the sistem gives back:

$$shape = \phi and scale = \mu/\phi$$

This will result useful later.

Also, $\mu = exp(\eta)$, where η are the linear predictors and $\phi = exp(\theta)$, note that the prior is set on θ.

The prior chosen for the precision parameter ϕ is a Gamma(1,0.01), trying to be non-informative. Then a log-gamma(1,0.01) is assigned to θ.

```
summary(mod_rainfall_lin_gamma)
```

```
## Time used:
##     Pre = 0.27, Running = 0.93, Post = 0.0636, Total = 1.26
## Fixed effects:
##                mean    sd 0.025quant 0.5quant 0.975quant  mode kld
## (Intercept)   0.802 0.015      0.774    0.802      0.831 0.802   0
## windspeed_s   0.033 0.015      0.004    0.033      0.061 0.033   0
## temperature_s 0.013 0.015     -0.016    0.013      0.043 0.013   0
## year_s        0.068 0.015      0.039    0.068      0.097 0.068   0
##
## Model hyperparameters:
##                                                mean   sd 0.025quant 0.5quant
## Precision-parameter for the Gamma observations 4.81 0.21       4.41     4.80
##                                                0.975quant mode
## Precision-parameter for the Gamma observations       5.23 4.80
##
## Watanabe-Akaike information criterion (WAIC) ...: 2689.75
## Effective number of parameters .................: 4.85
##
```

```
## Marginal log-Likelihood:  -1380.55
## CPO, PIT is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

High precision ϕ suggests low variance (variance $= 1/\tau = \mu^2/(s\phi)$) in the rainfall data, meaning predictions are relatively stable. In our model the precision is well-constrained within the credible interval [4.41, 5.23].

Since we are using a Gamma distribution :$\mu = exp(\eta)$, then μ is increasing in the β_i. Given the previous results, we expect positive β_i, that we find here (all the posterior mean of the β_i covariates in the Normal model, that uses a identity link function, were positive. We expect same behaviour here.)

As in the Gaussian model, the year and windspeed covariates are the most influent ones (year has higher mean value, but the two 95% CIs again overlap). Also, temperature does not seem to be statistically influent (CIs includes 0).

Let's now check goodness of fit:

```
#Compute goodness of fit criteria for Normal family
NSLCPO_norm <- -sum(log(mod_rainfall_lin_inla$cpo$cpo))
WAIC_norm <- mod_rainfall_lin_inla$waic$waic


#Compute goodness of fit criteria for Normal family
NSLCPO_wei <- -sum(log(mod_rainfall_lin_wei$cpo$cpo))
WAIC_wei <- mod_rainfall_lin_wei$waic$waic


#Compute goodness of fit criteria for Normal family
NSLCPO_gamma <- -sum(log(mod_rainfall_lin_gamma$cpo$cpo))
WAIC_gamma <- mod_rainfall_lin_gamma$waic$waic

cat("NSLCPO for Normal model:",NSLCPO_norm ,"\n")
```

```
## NSLCPO for Normal model: 1372.475
```

```
cat("NSLCPO for Weibull model:",NSLCPO_wei ,"\n")
```

```
## NSLCPO for Weibull model: 1353.7
```

```
cat("NSLCPO for Gamma model:",NSLCPO_gamma ,"\n\n\n")
```

```
## NSLCPO for Gamma model: 1344.873
```

```
cat("WAIC for Normal model:",WAIC_norm ,"\n")
```

```
## WAIC for Normal model: 2744.951
```

```
cat("WAIC for Weibull model:",WAIC_wei ,"\n")
```

```
## WAIC for Weibull model: 2707.396
```

```r
cat("WAIC for Gamma model:",WAIC_gamma ,"\n")
```

```
## WAIC for Gamma model: 2689.746
```

$NSLCPO = -\sum_1^n(\log(CPO_i))$. Where CPO_i measures how likely is the observation i given the rest of observations given the model ($CPO_i = p(y_i|y_{(-i)})$). So, smaller values of NSLCPO indicate better fit.

WAIC is a Bayesian model criteria that estimates a model's out of sample predictive accuracy by averaging over the entire posterior distribution while applying a penalty for model complexity: lower values indicating better fit. (it's a generalization for bayesian model for AIC of the linear frequentist models).

WAIC is similar to DIC but WAIC is fully bayesian, does not rely on point estimates but instead integrates over the posterior.

Since lower is the better for both NSLCPO and WAIC, it can be concluded that the Gamma is the best fit model. Also, the Weibull model performs better than the Normal one, but still Gamma fits better the data.

Pick best model and posterior predictive checks.

```r
#Fit the best model again (gamma) to predict

mod_rainfall_lin_gamma_pred <- inla(rainfall ~ windspeed_s +
                                    temperature_s + year_s,
                                    data= Edi.rain.clean, family='gamma',
                                    control.family = list(hyper= alpha_prior),
                                    control.fixed = beta_prior,
                                    control.compute = list(config=TRUE),
                                    verbose=TRUE)
```

```r
#Sampling
nsamp_pred=10000
samples_gamma <- inla.posterior.sample(n = nsamp_pred,
                                       result=mod_rainfall_lin_gamma_pred )
#Predictors:
pred_gamma <- inla.posterior.sample.eval(function(...) {Predictor},
                                         samples_gamma)

#Precision:
prec_gamma <- inla.posterior.sample.eval(function(...) {theta}, samples_gamma)

#Apply inverse of link function to get original scale
mu_samples <- exp(pred_gamma)

#Generate posterior predictive replicated data
yrep <- matrix(0, nrow=n, ncol=nsamp_pred) #note n is number of observation
                                           #in the dataset

for (i in 1:n) {
  #inla.doc("gamma")
  yrep[i, ] <- rgamma(nsamp_pred, shape= prec_gamma,
                      scale = (mu_samples[i, ])/prec_gamma)
}

yrep_min <- apply(yrep,2,min)
```

```
yrep_max <- apply(yrep,2,max)
yrep_median <- apply(yrep,2,median)

require(fBasics)
```

```
## Loading required package: fBasics
```

```
yrep_skewness <- apply(yrep,2,skewness)
yrep_kurtosis <- apply(yrep,2,kurtosis)

y<-Edi.rain.clean$rainfall #our response
```

Our best model from the previous point was the Gamma.

In order to perform posterior predictive checks, it's required to sample from the posterior predictive distribution on the observations that we know the real response of.

In order to do that the predictor variables, that ARE NOT samples from the posterior predictive, but are the linear predictors of our model,are sampled. The same thing happens for the precision (hyperparameter of the model with family Gamma).

Note: to compute the fitted values, we need to do $\exp(\eta)$, where η are the linear predictors that we obtain from the function inla.posterior.eval. We use exp() since the link function for the gamma family is log as default.

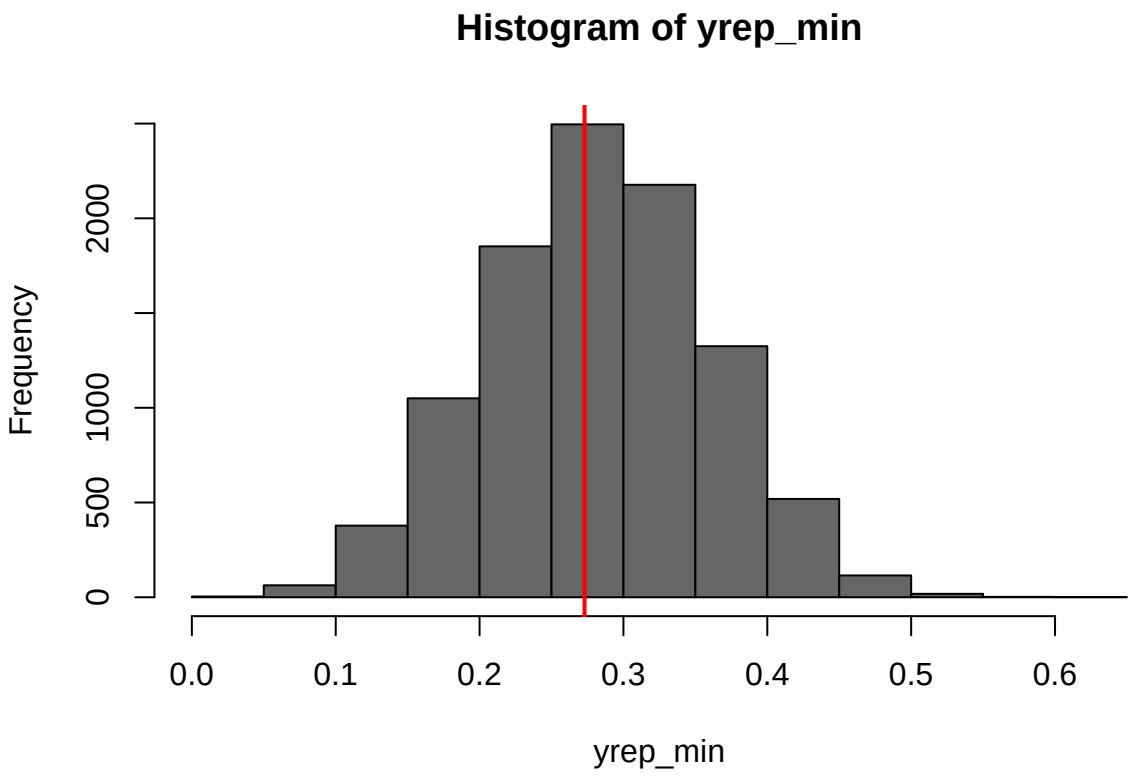
Note: after sampling we got a matrix of predictor variables (dimension: ($n \times n_{samp}$) where: n= number of observation in the dataset, n_{samp}= number of samples for each observation) and a vector (of length n) for the precision ϕ , hyperparameter of the Gamma model.

To get actual samples from the posterior predictive distribution (y_{rep}), now we need to sample from a gamma with shape = ϕ and scale = $exp(\eta)/\phi$. That is done in the for loop, using the function rgamma.

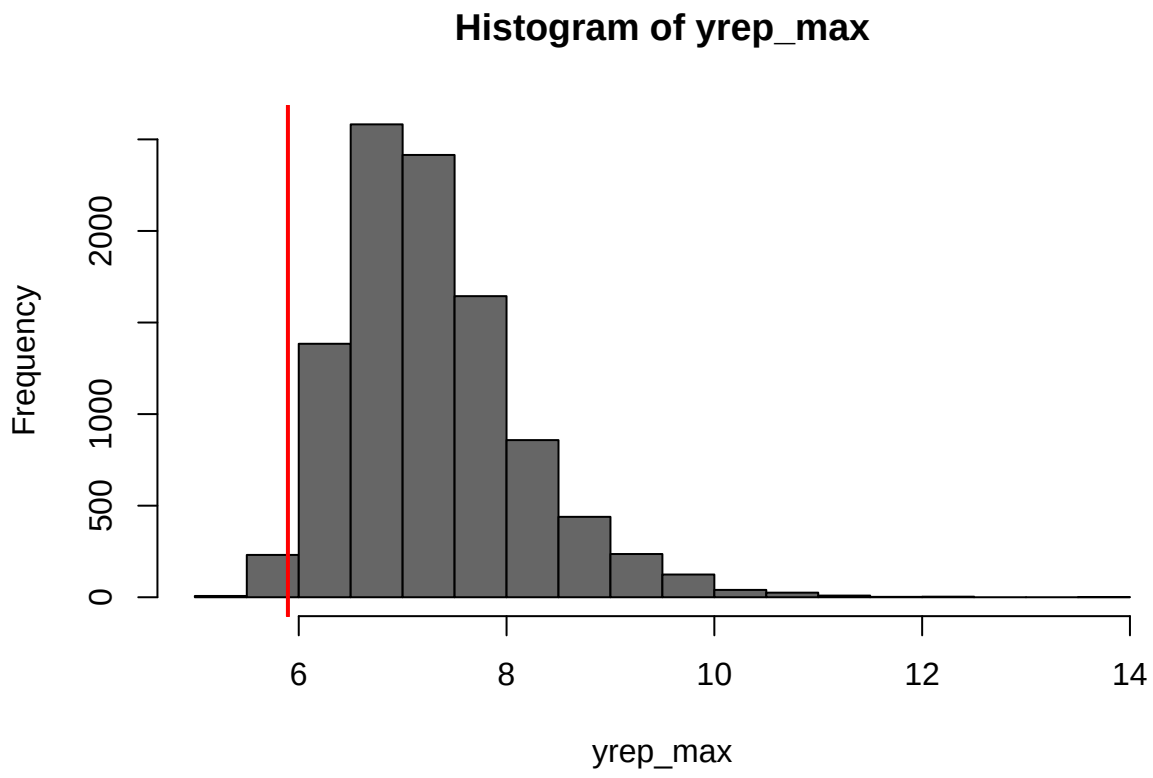
The formulas to compute the shape and scale in function of ϕ and η, have been found starting from the information contained in the inla.doc("gamma"). See earlier when the Gamma model was introduced.

Now we are left with a matrix y_{rep} of actual posterior predictive samples, where for each observation (from 1 to n) we have got n_{samp} samples. Using apply on the columns we get the min,max,median,skewness and kurtosis for each one of the n_{samp} samples, computed in order to compare our predictive distribution with the real data.

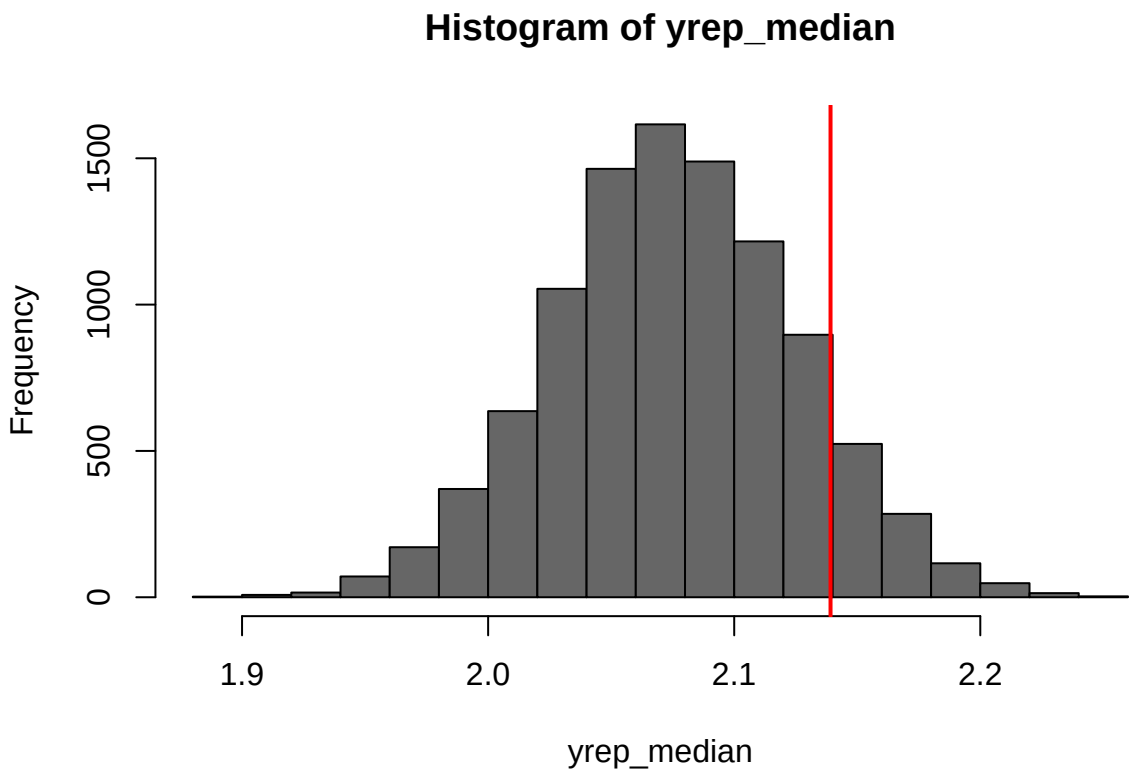
```
#minimum
hist(yrep_min,col="gray40")
abline(v=min(y),col="red",lwd=2)
```



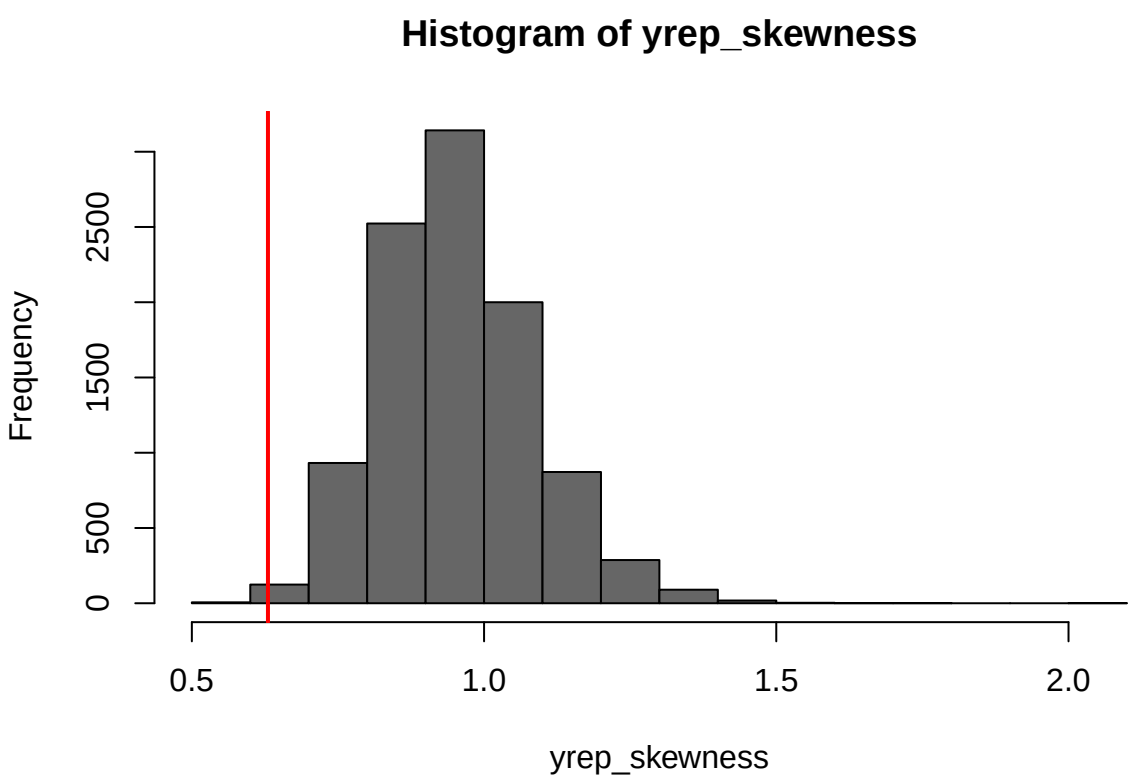

```
# maximum
hist(yrep_max,col="gray40")
abline(v=max(y),col="red",lwd=2)
```



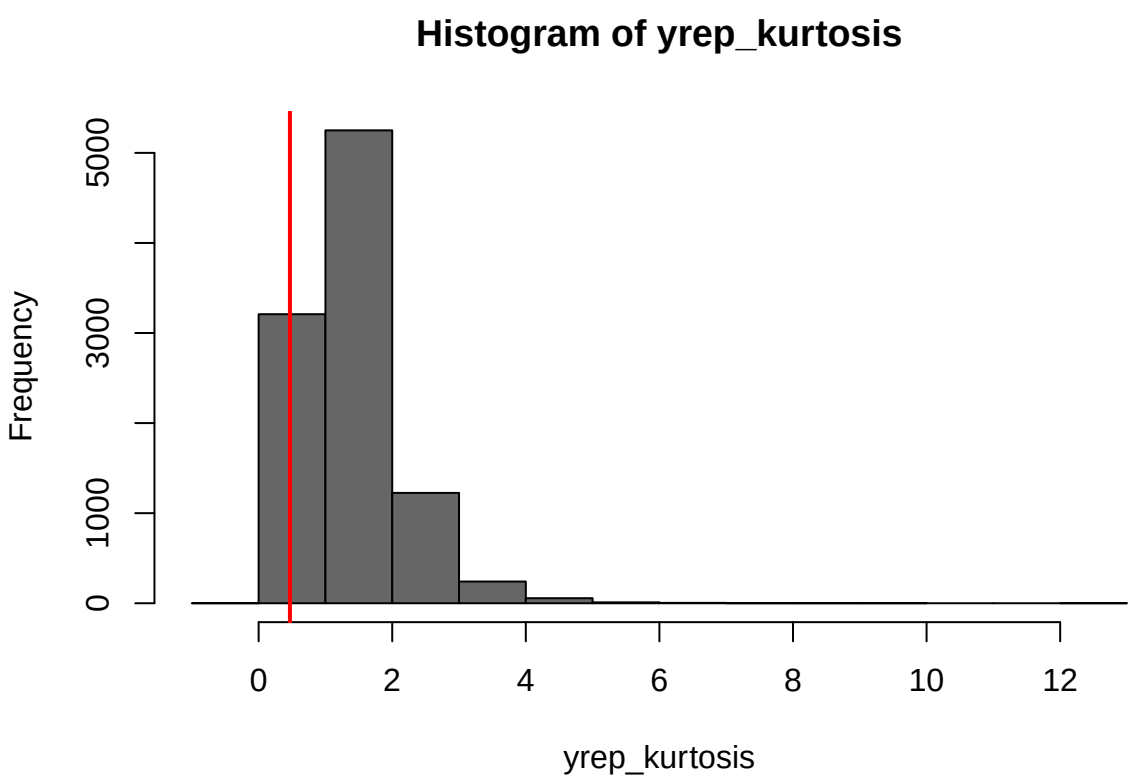

```
#median
hist(yrep_median,col="gray40")
abline(v=median(y),col="red",lwd=2)
```




```
#skewness
hist(yrep_skewness,col="gray40")
abline(v=skewness(y),col="red",lwd=2)
```




```
#kurtosis
hist(yrep_kurtosis,col="gray40")
abline(v=kurtosis(y),col="red",lwd=2)
```



The posterior predictive checks suggest that the Gamma model provides a reasonable fit to the data but there are some problems in capturing extreme values.

The minimum and median statistics allign well with the observed data, indicating that the model represents in a good way the central and lower part of the distribution.

On the other hand, the maximum values tend to be overestimated, suggesting that the model predicts more extreme high values than observed.

Similarly, the skewness and kurtosis indicate that the model expects a more pronounced skeweness and heavier tails than what is actually present in the data.

These results imply that while the Gamma model captures the overall distribution reasonably well, it may slightly overestimate variability and extreme events.

posterior predictive probability that the total rainfall in 2022 will exceed the previously recorded maximum of the annual total rainfall throughout the observation period.

```
#Standardise covariates on original database
Edi.rain$windspeed_s  <- scale(Edi.rain$windspeed, center=TRUE, scale=TRUE)
Edi.rain$temperature_s <- scale(Edi.rain$temperature,
                                center=TRUE, scale=TRUE)
Edi.rain$year_s  <- scale(Edi.rain$year, center=TRUE, scale=TRUE)

#Fit inla model (Gamma) with whole dataset

mod_rainfall_lin_gamma_pred22 <- inla(rainfall ~ windspeed_s +
                                      temperature_s + year_s,
                                    data= Edi.rain, family='gamma',
```

```
                                control.family = list(hyper= alpha_prior),
                                control.fixed = beta_prior,
                                control.compute = list(config=TRUE),
                                verbose=TRUE)
```

Since prediction is now going to be made on year 2022 the original dataset (Edi.rain) is used. We need to standardize the covariates again, since we had done it previously but in the dataset Edi.rain.clean.

The Gamma model is fitted again using the whole dataset.

Now we go on, computing the max rain in the previous years:

```
#Get max value in a year in the past (before 2022)

#Numbers of years in the dataset (excluding 2022)
n_years <- length(Edi.rain.clean$rainfall) / 12  # Assuming the dataset is complete

start_year <- 1940 #first year in the dataset

#Reshape the vector into a matrix with 12 rows (months) and n_years columns
rainfall_matrix <- matrix(Edi.rain.clean$rainfall, nrow=12, byrow=FALSE)

#ignoring leap years
days_in_months <- c(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)

#Average yearly rainfall for each year
average_yearly_rainfall <- apply(rainfall_matrix, 2,
                                          function(x) sum(x * days_in_months))

#Compute max in the previous years
max_rain_general <- max(average_yearly_rainfall)#including leap years
```

To get the posterior probability of exceeding the maximum of rain happened in one year, the computation for this max value is needed. The yearly rain is computed as the sum of the multiplication between the daily rain in each month by the number of days in the month (all years are considered to have 365 days).

After that a matrix with 12 rows (months) and 81 columns (years from 1940 to 2021) is created and all the data about the rain in each are stored inside.

The amount of yearly rain is computed using apply on the columns, summing the multiplications of the daily mean rain by the number of days in each month. Then the max from the vector of yearly rain in each year is computed.

```
nsamp22 <- 10000

#Get rows for prediction of 2022
print(nrow(Edi.rain))
```

```
## [1] 996
```

```
print(nrow(Edi.rain.clean))
```

```
## [1] 984
```

```
samp22 <- inla.posterior.sample(n = nsamp22,
                                result = mod_rainfall_lin_gamma_pred22 ,
                                selection = list(Predictor=985:996))

#Get linear predictors
predictor_samples22 <- matrix(unlist(lapply(samp22, function(x)
                                    (x$latent[1:12]))),
                              nrow=12,ncol=nsamp22 )

#Get mu samples
mu_samples22 <- exp(predictor_samples22)

#Get precision samples
prec_samp22 <-unlist(lapply(samp22, function(x)(x$hyperpar[1])))

post_pred22 <- matrix(0, 12, nsamp22)

#Posterior predictive samples
for(i in 1:12){
post_pred22[i,] <- rgamma(nsamp22, shape= prec_samp22,
                          scale = mu_samples22[i,] / prec_samp22)
}

#Rain for each year in each sample
year22_rain <- apply(post_pred22, 2, function(x) sum(x * days_in_months))
```

To compute the posterior probability, we need to obtain posterior predictive samples for the year 2022. To select only samples for these observation we select the last 12 rows of the dataset (from 985 to 996).

In order to do that, as done previously, we sample the linear predictors and transform them in order to get the fitted values.

Also, we get samples of the precision ϕ . After this, we are ready to sample from the posterior predictive using the rgamma function as before.

The matrix post_pred22, is of dimension $12 \times n_{samp22}$: each column is a sample for rain that will happen in 2022.

Note: each column is a sample of one year of rain, to compute the yearly rain we apply the same method that we used previously. All this is done using the apply function on the columns. Obtaining a vector year22_rain containing n_{samp22} posterior predictive samples of yearly rainfall in 2022.

Now we can compute the probability:

```
#NOT ignoring leap years in the search for the max
exceed22 <- year22_rain > max_rain_general
post_prob <- mean(exceed22)

cat('The posterior probability of rain in 2022 exceeding the max of the
    previous years is:', post_prob)
```

```
## The posterior probability of rain in 2022 exceeding the max of the
##     previous years is: 0.1432
```